

Indian Institute of Technology , Jodhpur



Assignment No 1

Resilient State Machine Replication:

Paxos & PBFT

Submitted By: Vinay Singh

Roll No : G25AI1054

GIT Repo [Link](https://github.com/vinprac04/distributed-consensus-engine.git) (<https://github.com/vinprac04/distributed-consensus-engine.git>)

Video [Link](#) : 📺 FDA Assignment 1.mov

Table of Content

1. Assignment Summary
2. Assignment Objective
3. System Architecture
4. Tech Stack
5. Algorithms Used
 - Leader Election (Bully Algorithm)
 - Paxos Algorithm
 - Practical Byzantine Fault Tolerance (PBFT)
6. Complete Orchestration
7. Code Explanation
 - node.py
 - adversary.py
 - client.py
 - crypto_utils.py
 - Docker Compose
8. Docker Architecture
9. Chaos Testing
10. Results and Observations
11. Conclusion

1. Assignment Summary

In this assignment we have been asked to design and implement a Distributed

Consensus Engine which is capable of holding and maintaining a consistent transaction ledger among five distributed nodes.

As each node will operate independently they all must agree with each other before any transaction log is stored in the system

In this system we have five distributed nodes that work together to process client transactions while maintaining consistency and fault tolerance.

The architecture supports two different operational modes:

- Mode A (Crash Fault Tolerance) using Leader Election and Paxos
- Mode B (Byzantine Fault Tolerance) using PBFT

In this project we will demonstrate how distributed systems maintain reliability even when nodes have faults and crashes.

Environment setup is done using Docker containers.

2. Assignment Objective

Main objective of this project is to build a distributed transaction ledger where multiple nodes maintain the same sequence of committed transactions. Every transaction should be recorded consistently across all honest nodes so that the ledger remains synchronized and reliable.

Second objective is fault tolerance. If any nodes crash, the remaining nodes should still be able to reach consensus and continue processing transactions while maintaining the ledger consistency.

Third objective, is to handle malicious node handling (Byzantine faults). If any node sends incorrect messages, the honest nodes must detect such behavior, reject invalid information, and ensure that only verified transactions are committed to the ledger.

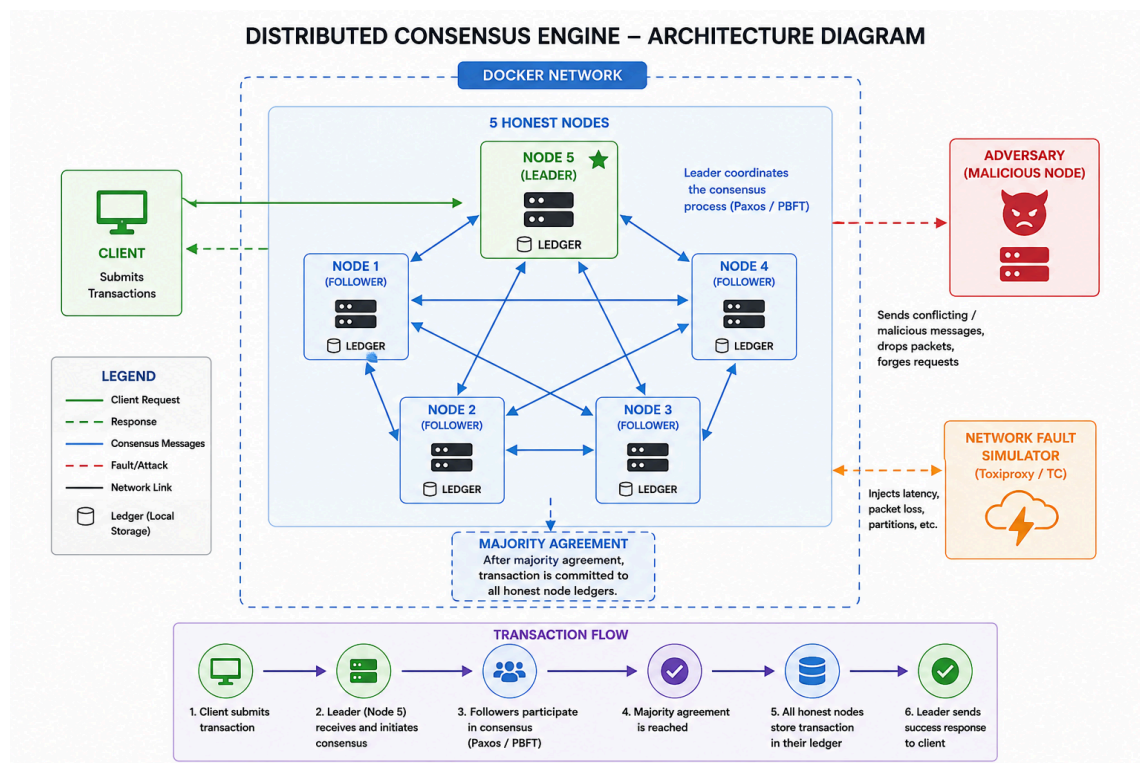
Overall, the goal is to develop a reliable, fault-tolerant, and secure distributed consensus system that maintains a consistent ledger while handling both crash failures and malicious attacks.

3. System Architecture

The architecture consists of:

- 5 server nodes
- 1 client
- 1 Adversary (Malicious) Node
- Docker Network
- Network Fault Simulator

Client system sends transactions to the leader node. Selected leader will own and coordinates the consensus process. Once majority consensus reached, all server nodes store the transaction into their ledger.



4. Tech Stack used Used

Technology	Purpose
Python	Implementation Language
Socket Programming	Node Communication
Docker	Containerization
Docker Compose	Cluster Orchestration
Cryptography Library	Digital Signatures
JSON	Message Format
Toxiproxy / Network Simulation	Chaos Testing

5. Algorithms Used and Implemented

5.1 Leader Election (Bully Algorithm)

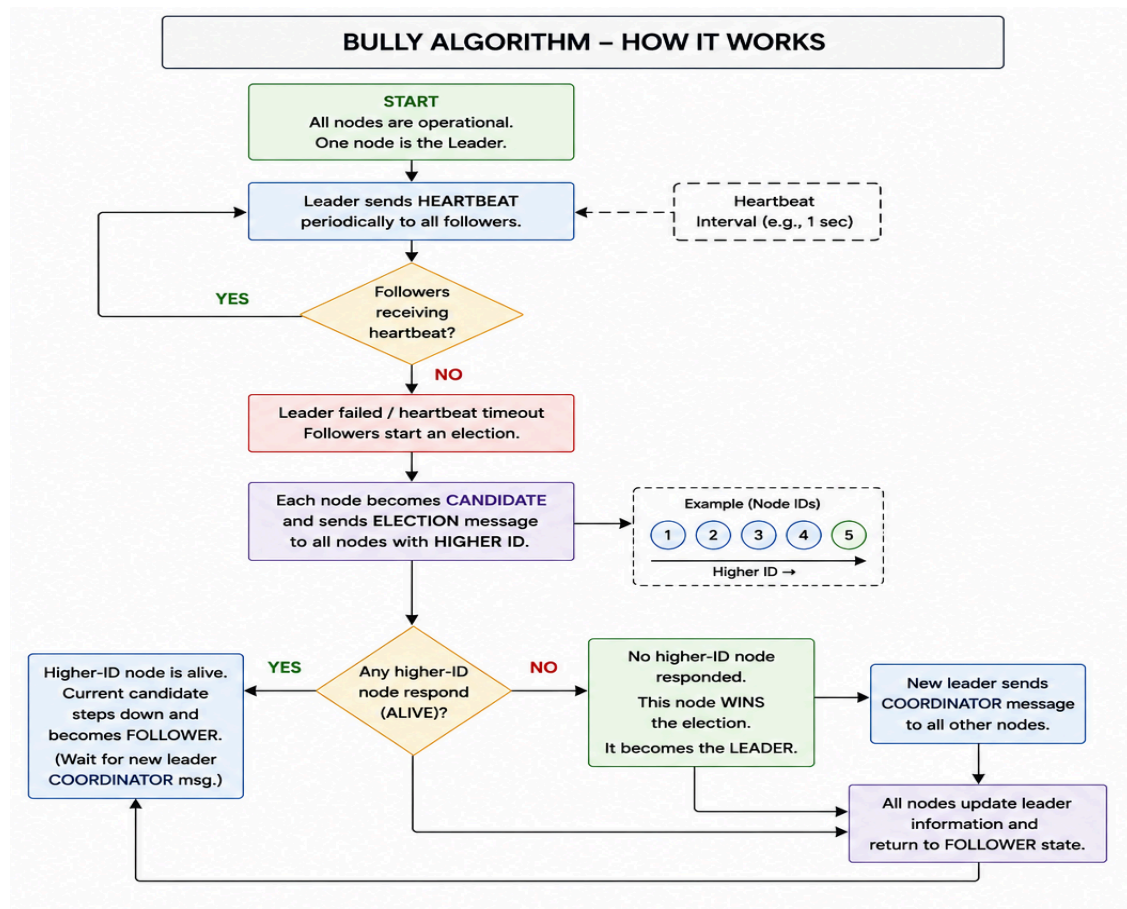
In distributed system setup, there is a need of one master/coordinator node to organize consensus.

Without a leader, multiple nodes may attempt to process the same transaction simultaneously, causing conflicts. Hence it is important and critical to have a elected master/leader node.

To achieve this Each node has been assigned unique ID.

When the leader fails:

- Followers stop receiving heartbeats.
- They initiate an election.
- Lower-ID nodes ask higher-ID nodes whether they are alive.
- If no higher-ID node responds, the current node becomes the leader.
- The new leader informs every other node.



5.2 Paxos Algorithm

Paxos is consensus algorithm which enables fault tolerance and consistent in transaction among the distributed nodes ledgers.

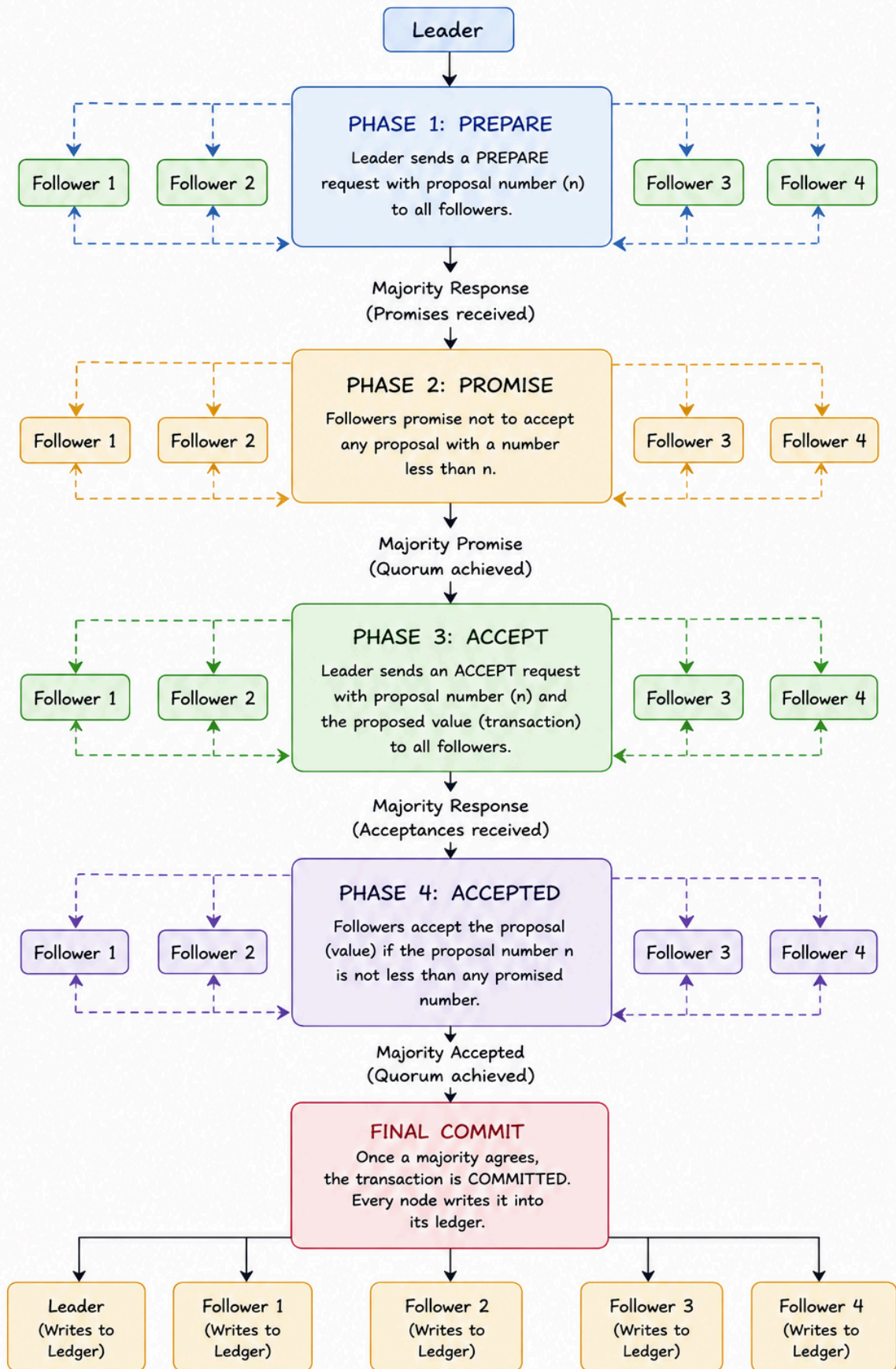
It has four key phases

Prepare → Promise → Accept → Accepted

Paxos Phases

Phase	Description	Purpose
Phase 1: Prepare	The leader (proposer) sends a Prepare request with a unique proposal number to all follower nodes, asking whether it can propose a transaction.	To initiate the consensus process and notify all nodes about a new proposal while preventing conflicts with older proposals.
Phase 2: Promise	Each follower checks the proposal number and sends a Promise message, agreeing not to accept any proposal with a lower number.	To ensure that only the latest valid proposal proceeds, maintaining consistency across the cluster.
Phase 3: Accept	After receiving promises from a majority of nodes, the leader sends an Accept request containing the actual transaction value.	To request the participating nodes to accept the proposed transaction after obtaining majority support.
Phase 4: Accepted	Followers verify the request and, if valid, send an Accepted response back to the leader indicating their approval.	To confirm that the transaction has been accepted by the participating nodes.
Final Commit	Once the leader receives Accepted responses from a majority of nodes, consensus is achieved and all honest nodes write the transaction into their local ledger.	To ensure that every node stores the same committed transaction, maintaining a consistent and reliable distributed ledger.

PAXOS PHASES (Simplified)



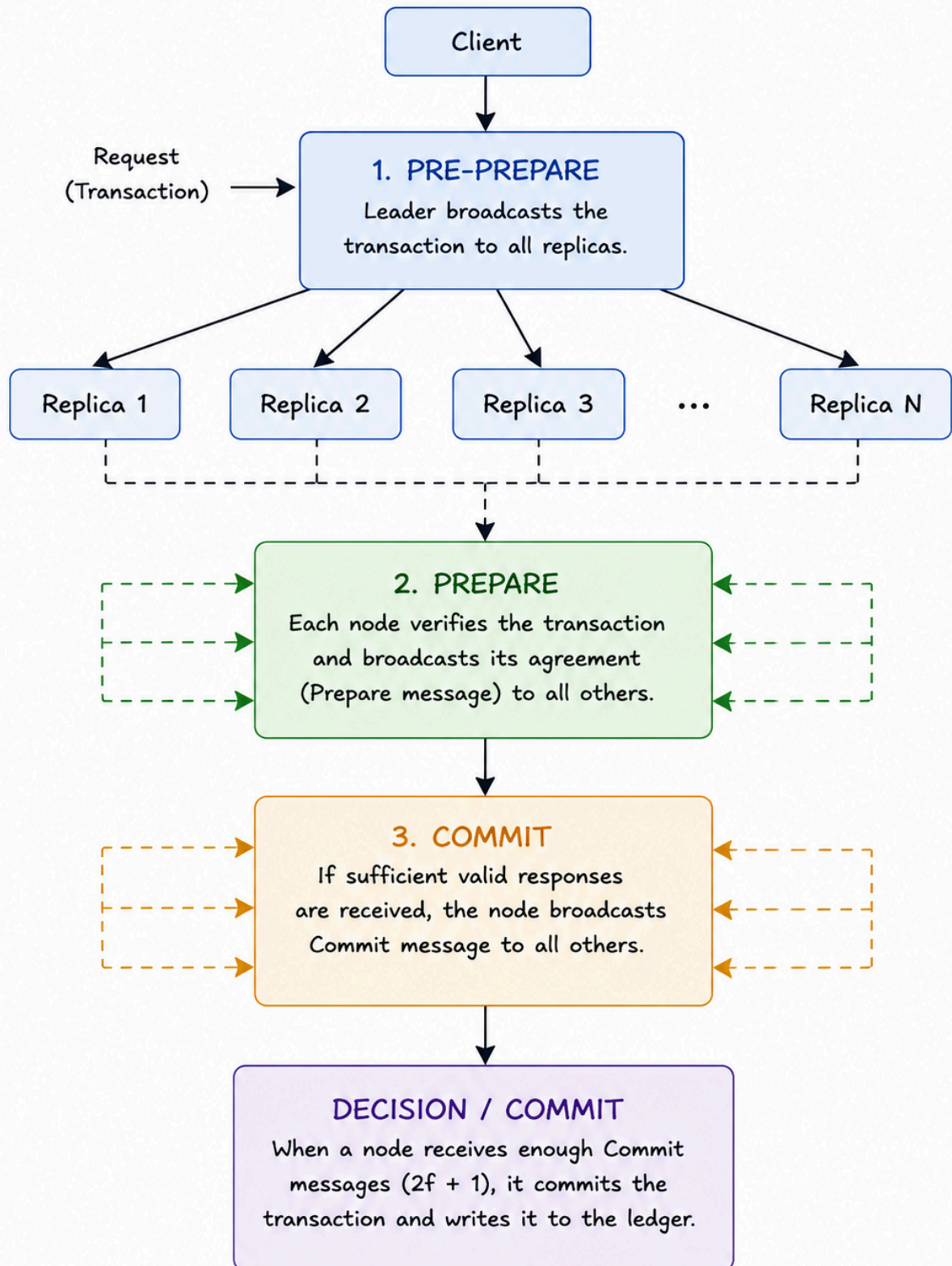
5.3 Practical Byzantine Fault Tolerance (PBFT)

Practical Byzantine Fault Tolerance (PBFT) is also consensus algorithm designed to maintain consistency in distributed systems even when some nodes behave maliciously.

PBFT algo has three phase to ensuring that only verified transactions are committed by all honest nodes. Pre-Prepare → Prepare → and Commit

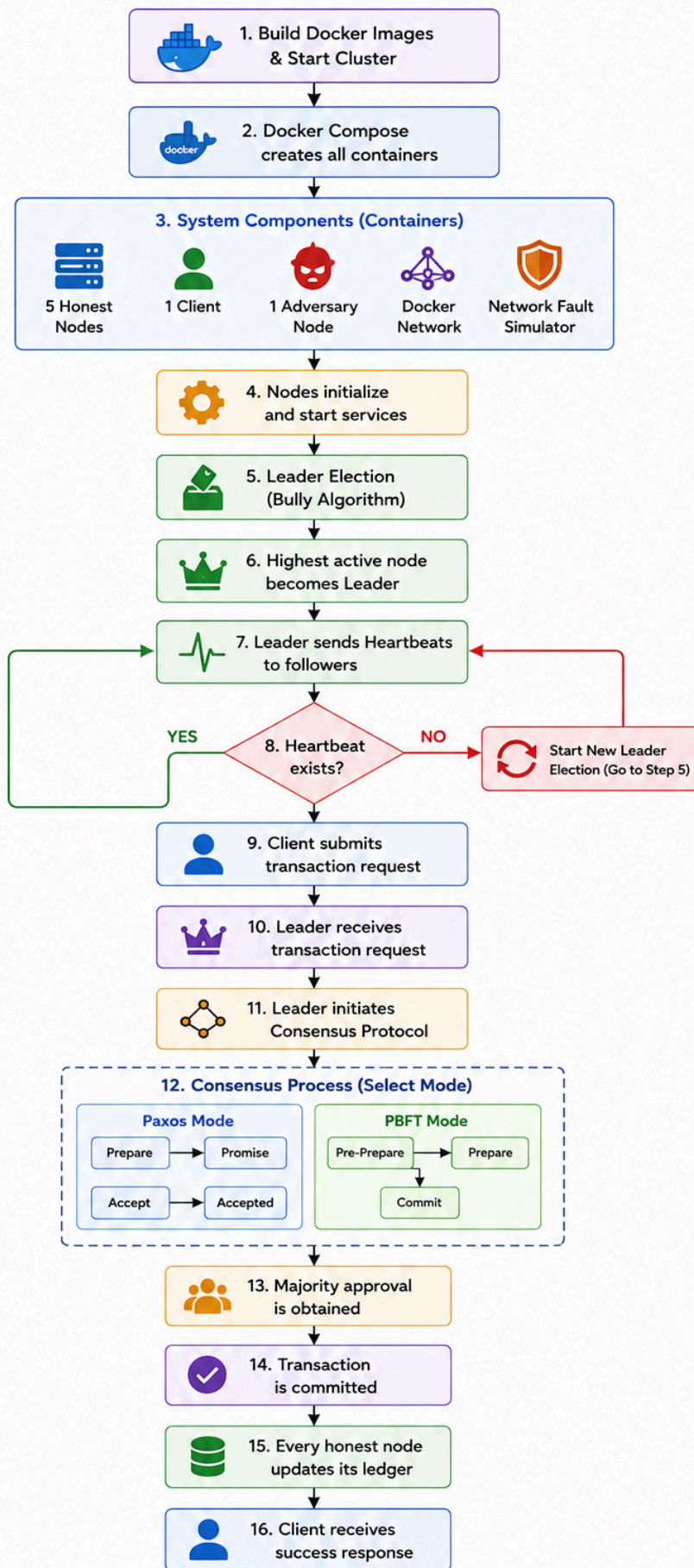
Phase	What Happens	Purpose
Phase 1: Pre-Prepare	The leader (Primary) receives the client request and broadcasts a Pre-Prepare message containing the transaction and sequence number to all replica nodes.	To initiate the consensus process and ensure all nodes receive the same transaction request.
Phase 2: Prepare	Each node verifies the received transaction (message integrity, sequence number, etc.). If valid, it broadcasts a Prepare message to the other nodes indicating its agreement.	To validate the transaction and achieve agreement among the majority of nodes before committing it.
Phase 3: Commit	After receiving sufficient valid Prepare messages, each node broadcasts a Commit message. Once enough Commit messages are received, the transaction is finalized and written to the ledger.	To ensure that consensus has been reached and that all honest nodes permanently commit the same transaction to their ledger.

PBFT PHASES



6. Complete Orchestration of designed system

1. **Build Docker Images & Start Cluster:** The application is first packaged into Docker images and the distributed cluster is started, preparing the environment for execution.
2. **Docker Compose Creates All Containers:** Docker Compose automatically creates and connects all required containers, including the five honest nodes, client, adversary node, and network components.
3. **System Components Initialization:** Each node initializes its communication services, consensus module, cryptographic utilities, and local ledger before joining the cluster.
4. **Nodes Initialize and Start Services:** After initialization, all nodes start their services and establish communication with peer nodes in the distributed network.
5. **Leader Election (Bully Algorithm):** The Bully Algorithm is executed to elect a leader, where the highest active node is selected to coordinate the system.
6. **Highest Active Node Becomes Leader:** Once elected, the leader becomes responsible for receiving client requests and managing the consensus process.
7. **Leader Sends Heartbeats to Followers:** The leader periodically sends heartbeat messages to all follower nodes to indicate that it is active and functioning normally.
8. **Heartbeat Validation:** Followers continuously monitor these heartbeat messages, and if they stop receiving them, a new leader election process is initiated automatically.
9. **Client Submits Transaction Request:** The client sends a transaction request to the elected leader for processing and consensus.
10. **Leader Receives Transaction Request:** The leader validates the received request and prepares it for execution through the appropriate consensus protocol.
11. **Leader Initiates Consensus Protocol:** Depending on the operating mode, the leader starts either the Paxos or PBFT consensus algorithm to coordinate agreement among nodes.
12. **Consensus Process (Paxos/PBFT):** All participating nodes exchange protocol-specific messages, verify the transaction, and work together to reach a common agreement.
13. **Majority Approval is Obtained:** The transaction proceeds only after the required majority of nodes approves the proposal, ensuring consistency and fault tolerance.
14. **Transaction is Committed:** Once consensus is achieved, the transaction is officially committed and considered final.
15. **Every Honest Node Updates Its Ledger:** All honest nodes append the committed transaction to their local ledger, ensuring that every node maintains the same transaction history.
16. **Client Receives Success Response:** Finally, the leader sends a success response back to the client, confirming that the transaction has been processed and committed successfully.



7. Important Code Explanation

7.1 node.py

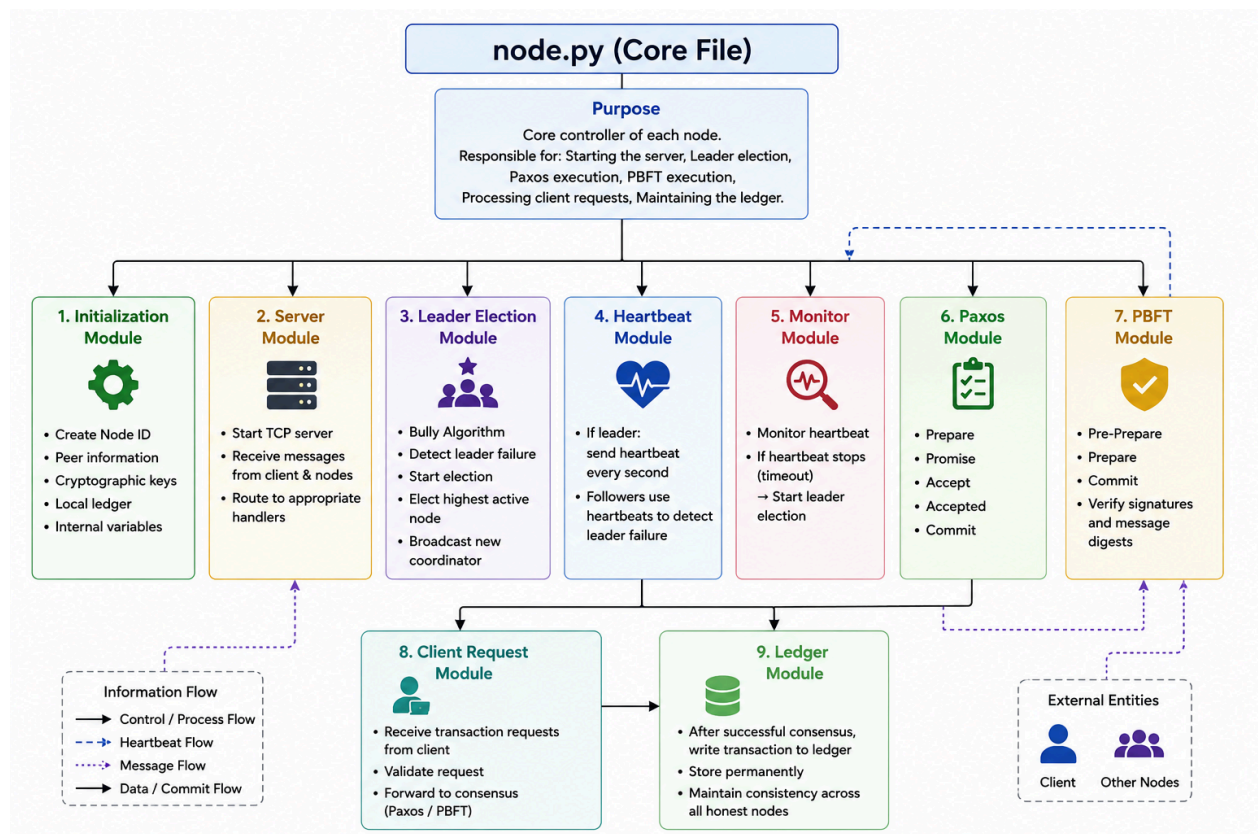
node.py is the core file of the system and serves as the central controller for each node in the cluster. It is responsible for:

- Starting the server
- Leader election
- Paxos execution
- PBFT execution
- Processing client requests
- Maintaining the ledger

Major Components

Module	Description
Initialization Module	Initializes the node by creating the Node ID, peer information, cryptographic keys, local ledger, and other internal variables required for execution.
Server Module	Starts the TCP server, receives messages from clients and other nodes, and routes them to the appropriate message handlers based on their type.
Leader Election Module	Implements the Bully Algorithm to elect a leader, detect leader failures, and coordinate the election of a new leader when required.
Heartbeat Module	If the node is the leader, it periodically sends heartbeat messages to follower nodes to indicate that it is active and operational.
Monitor Module	Continuously monitors incoming heartbeat messages. If heartbeats are not received within the configured timeout, it automatically initiates a new leader election.
Paxos Module	Executes the Paxos consensus algorithm by implementing the Prepare, Promise, Accept, Accepted, and Commit phases to achieve crash fault tolerance.

PBFT Module	Executes the PBFT consensus algorithm through the Pre-Prepare, Prepare, and Commit phases while verifying digital signatures and message digests to handle Byzantine faults.
Client Request Module	Receives transaction requests from the client, validates them, and forwards them to the appropriate consensus protocol for processing.
Ledger Module	After successful consensus, permanently stores the committed transaction in the local ledger and maintains consistency across all honest nodes.



7.2 adversary.py

adversary.py is a specialized module that simulates a malicious (Byzantine) node within the distributed system. It intentionally violates the consensus protocol by sending conflicting or forged messages and suppressing communication, allowing the system to test its ability to detect and tolerate Byzantine faults during PBFT execution.

7.3 client.py

client.py acts as the transaction generator of the system. It identifies the current leader node, submits transaction requests, receives responses after consensus, and displays the final commit status to demonstrate successful transaction processing.

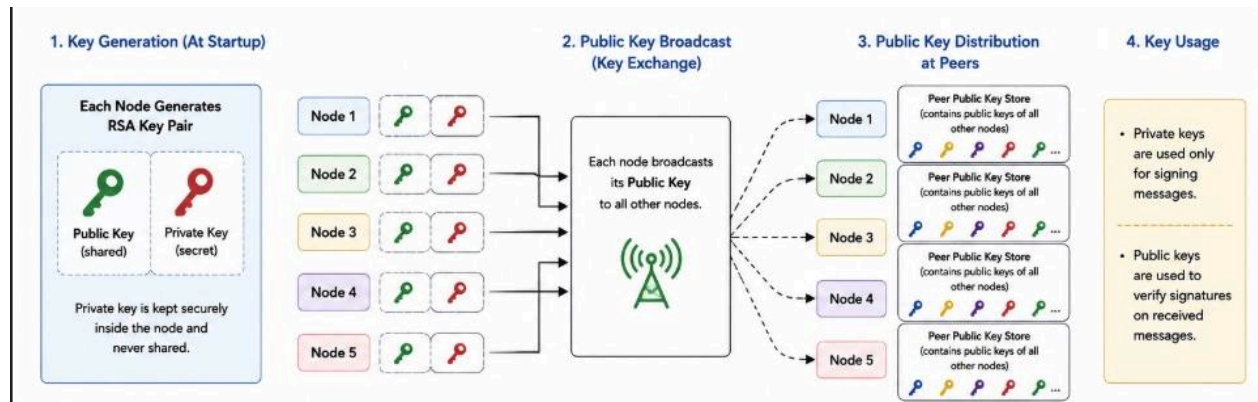
7.4 crypto_utils.py

crypto_utils.py provides the cryptographic security layer required for PBFT communication. It generates RSA key pairs, signs messages, verifies digital signatures, and computes SHA-256 hashes to ensure message authenticity, integrity, and protection against tampering.

Cryptographic Key Distribution in PBFT

In the PBFT implementation, each node generates its own **RSA public-private key pair** during startup. The **public key** is shared with all other nodes through a key exchange process, while the **private key** remains securely stored within the node and is never shared.

During consensus, every PBFT message is digitally signed using the sender's private key. Receiving nodes verify the signature using the stored public key and validate the **SHA-256 message digest**. Only messages with valid signatures and matching digests are accepted, ensuring authentication, integrity, and protection against malicious or tampered messages.



7.5 docker-compose.yml

docker-compose.yml is responsible for deploying and orchestrating the complete distributed environment. It creates and connects five honest nodes, one client, one adversary node, and the shared Docker network, providing a unified platform for consensus execution and network fault simulation.

8. Code Execution Snapshots

Execution Steps

Step 1: Build Docker Images

Build all Docker images required for the distributed consensus engine.

```
docker-compose build
```



```
-consensus-engine
CHAT  CLAUDE CODE
v TERMINAL  docker-compose + - - -
(base) vinaysingh@Vinays-Mac-mini distributed-consensus-engine % docker-compose build
=> => exporting config sha256:61e71d500233ff5288f5430c90642cd09744f15b147d3c0188a03 0.0s
=> => exporting attestation manifest sha256:a1885e80b8551cbee7b6d39b3f9055df50a773f 0.0s
=> => exporting manifest list sha256:58981e8dd37ae7bc02310dac27484b1557ce8f4c0a6849 0.0s
=> => naming to docker.io/library/distributed-consensus-engine-adversary:latest 0.0s
=> => unpacking to docker.io/library/distributed-consensus-engine-adversary:latest 0.0s
=> [client] exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:76b71ff7c23dfaee2279033b98742c281b9457a85d3d6e9d935 0.0s
=> => exporting config sha256:1eda46e2df014f3bc3c734c05669b4f60c8fa98bc4e8923404356 0.0s
=> => exporting attestation manifest sha256:f67db9bddcd8cfc947edd6435e1b6ea12e348a5 0.0s
=> => exporting manifest list sha256:343653293243c4dc895d6fef50f6fa957e187c7c7fa97c 0.0s
=> => naming to docker.io/library/distributed-consensus-engine-client:latest 0.0s
=> => unpacking to docker.io/library/distributed-consensus-engine-client:latest 0.0s
=> [node1] exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:dd46c233584873a4ee7df55f3025fdf41b9f41d1306cf28ecff 0.0s
=> => exporting config sha256:a1de3dab95b312897c26422f27d3025167f465c1f58a27aad734 0.0s
=> => exporting attestation manifest sha256:c319de631028b4f256954bf6fbf0187b58e1d0f 0.0s
=> => exporting manifest list sha256:c572b079d9d64bc225e038276e66bfde0166a9d1059188 0.0s
=> => naming to docker.io/library/distributed-consensus-engine-node1:latest 0.0s
=> => unpacking to docker.io/library/distributed-consensus-engine-node1:latest 0.0s
=> [node5] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:de9df14ebc6be00d3b82e8feae541103e73735d18929b5e2f4d 0.0s
=> => exporting config sha256:a48eae40477c8b86d47d43ea69cb7289ff9e6ff2ab252f49ac7df 0.0s
=> => exporting attestation manifest sha256:6703cc3c875a0262d7ba24d8337717cf4b92ee5 0.0s
=> => exporting manifest list sha256:78053f2b96706b54e1ebbbe64e2b0233bcc472a954a9 0.0s
=> => naming to docker.io/library/distributed-consensus-engine-node5:latest 0.0s
=> => unpacking to docker.io/library/distributed-consensus-engine-node5:latest 0.0s
=> [node3] resolving provenance for metadata file 0.1s
=> [node4] resolving provenance for metadata file 0.1s
=> [node1] resolving provenance for metadata file 0.1s
=> [node2] resolving provenance for metadata file 0.0s
=> [adversary] resolving provenance for metadata file 0.0s
=> [node5] resolving provenance for metadata file 0.0s
=> [client] resolving provenance for metadata file 0.0s
[+] build 7/7
✓ Image distributed-consensus-engine-node4 Built 2.4s
✓ Image distributed-consensus-engine-node2 Built 2.4s
✓ Image distributed-consensus-engine-node1 Built 2.4s
✓ Image distributed-consensus-engine-node3 Built 2.4s
✓ Image distributed-consensus-engine-node5 Built 2.4s
✓ Image distributed-consensus-engine-client Built 2.4s
✓ Image distributed-consensus-engine-adversary Built 2.4s
o (base) vinaysingh@Vinays-Mac-mini distributed-consensus-engine % docker-compose up
[+] up 9/9
✓ Network distributed-consensus-engine_consensus-net Created 0.0s
✓ Container node1 Created 0.1s
✓ Container node4 Created 0.1s
✓ Container node3 Created 0.1s
✓ Container node5 Created 0.1s
✓ Container toxiproxy Created 0.1s
✓ Container node2 Created 0.1s
✓ Container adversary Created 0.1s
✓ Container client Created 0.1s
Attaching to adversary, client, node1, node2, node3, node4, node5, toxiproxy
```

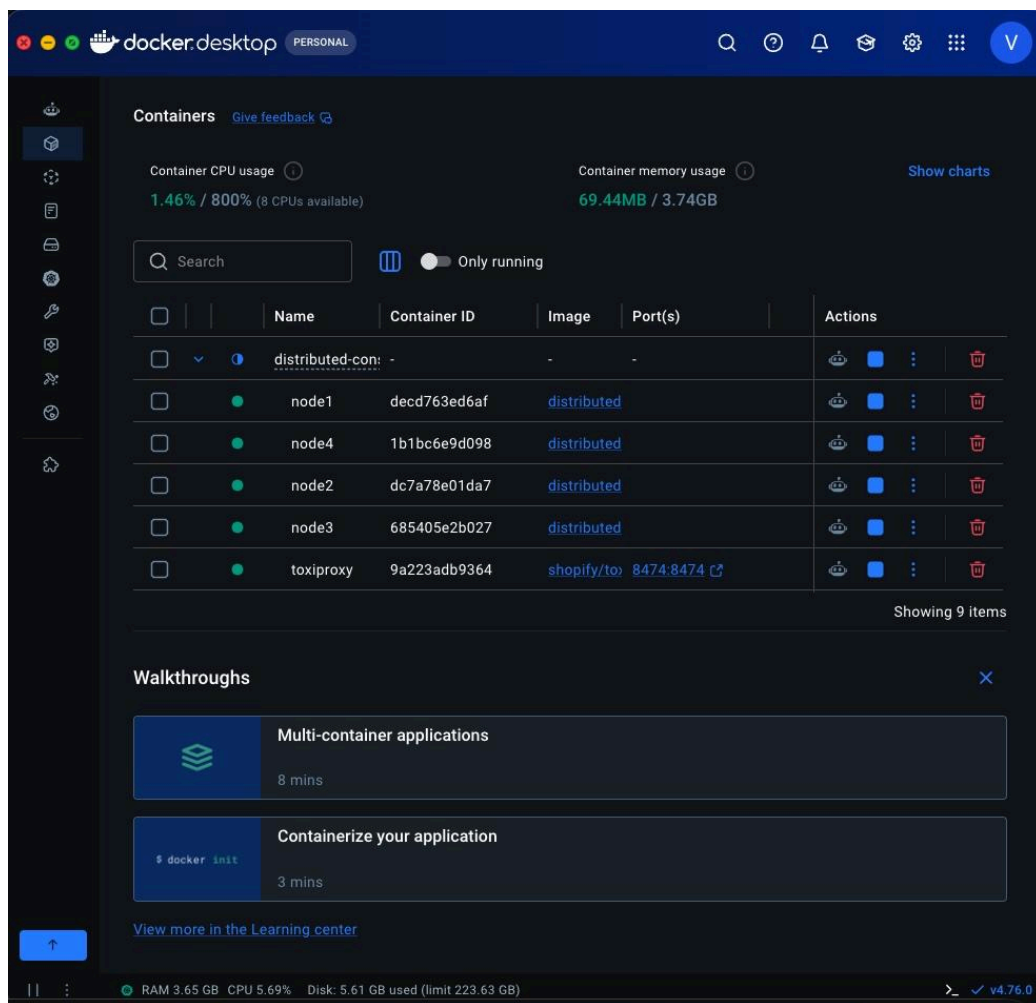

Step 2: Start the Cluster

Start all containers, including the five nodes, client, adversary node, and supporting services.

`docker-compose up`

or

`docker-compose up -d`



Step 3: Verify Running Containers

Verify that all containers are running successfully.

```
docker-compose ps
```

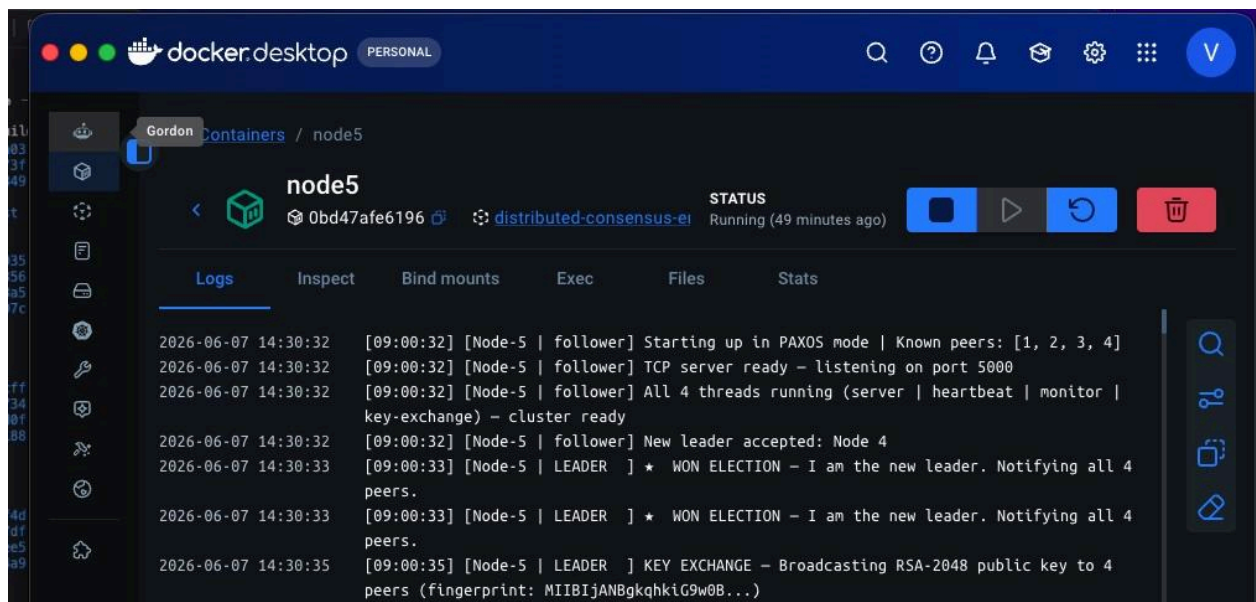
Step 4: Leader Election

Observe the Bully Algorithm electing the highest active node as the leader.

```
docker-compose logs -f | grep -E "LEADER|Leader"
```

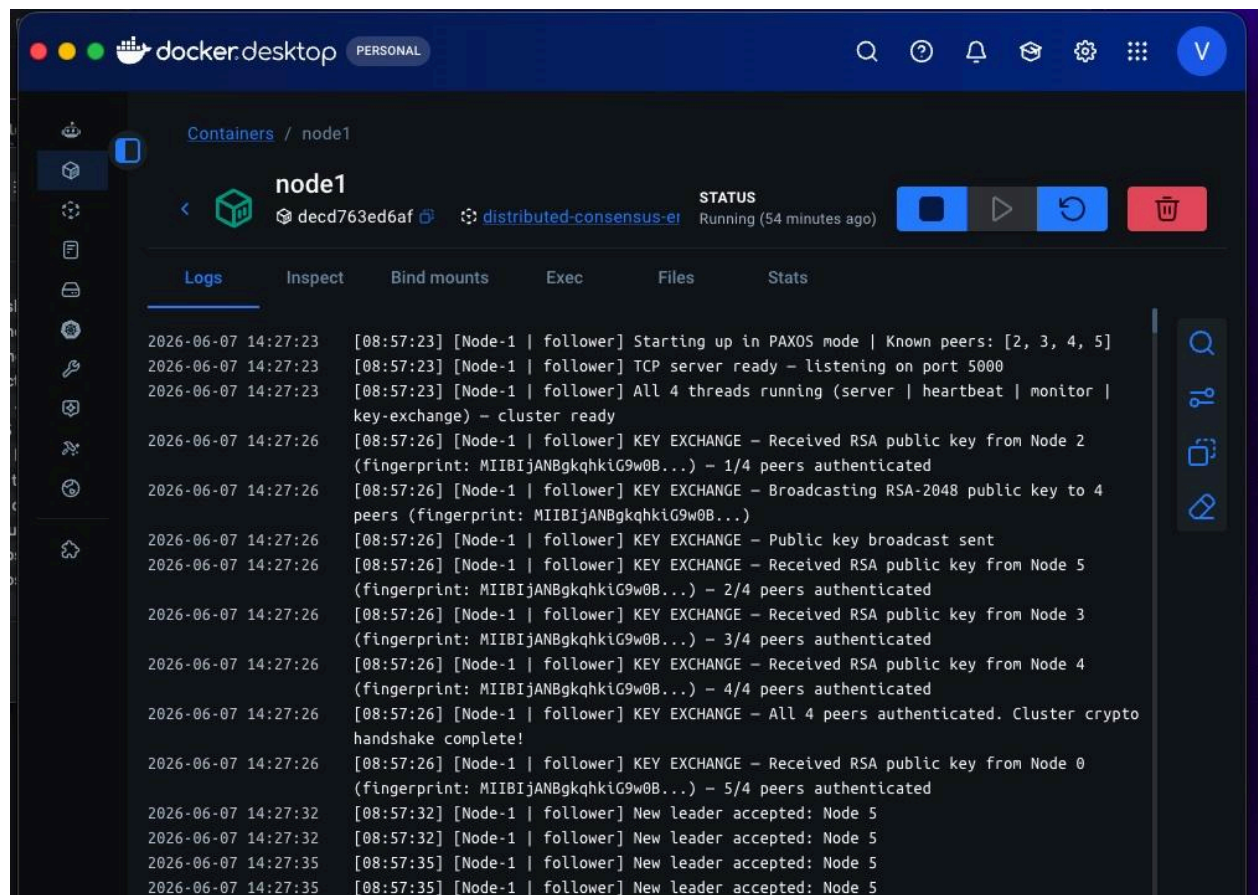
or

```
docker logs node5
```



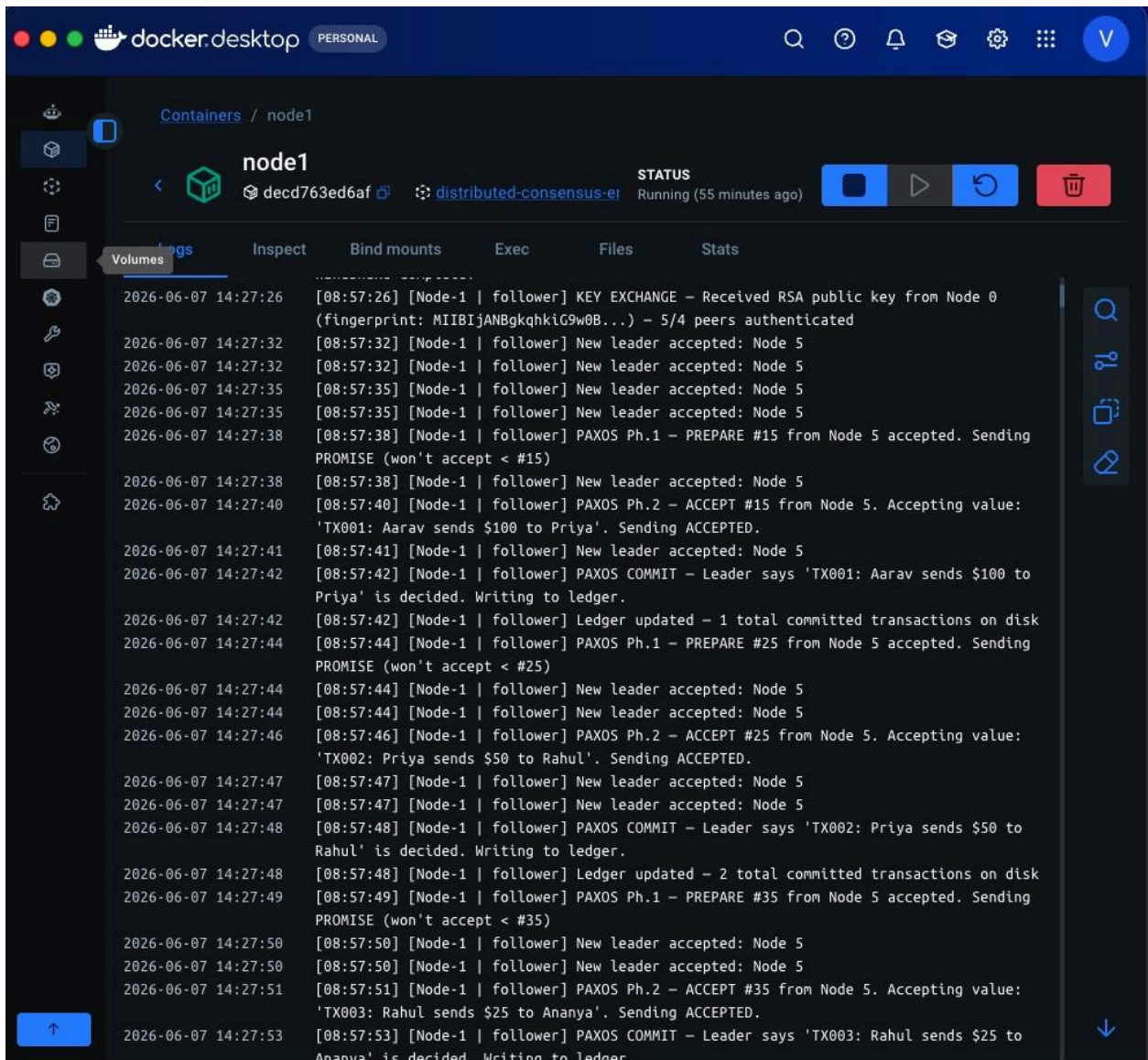
Step 5: Heartbeat Communication

Verify that the leader periodically sends heartbeat messages to follower nodes.



Step 6: Paxos Consensus Execution

Observe the Paxos phases (Prepare, Promise, Accept, Accepted, and Commit) during transaction processing.



Step 7: Transaction Commitment

Verify that transactions are successfully committed after majority consensus.

docker desktop PERSONAL

Containers / node1

node1

decd763ed6af distributed-consensus-e

STATUS Running (56 minutes ago)

Logs Inspect Bind mounts Exec **Files** Stats

Hide file editor

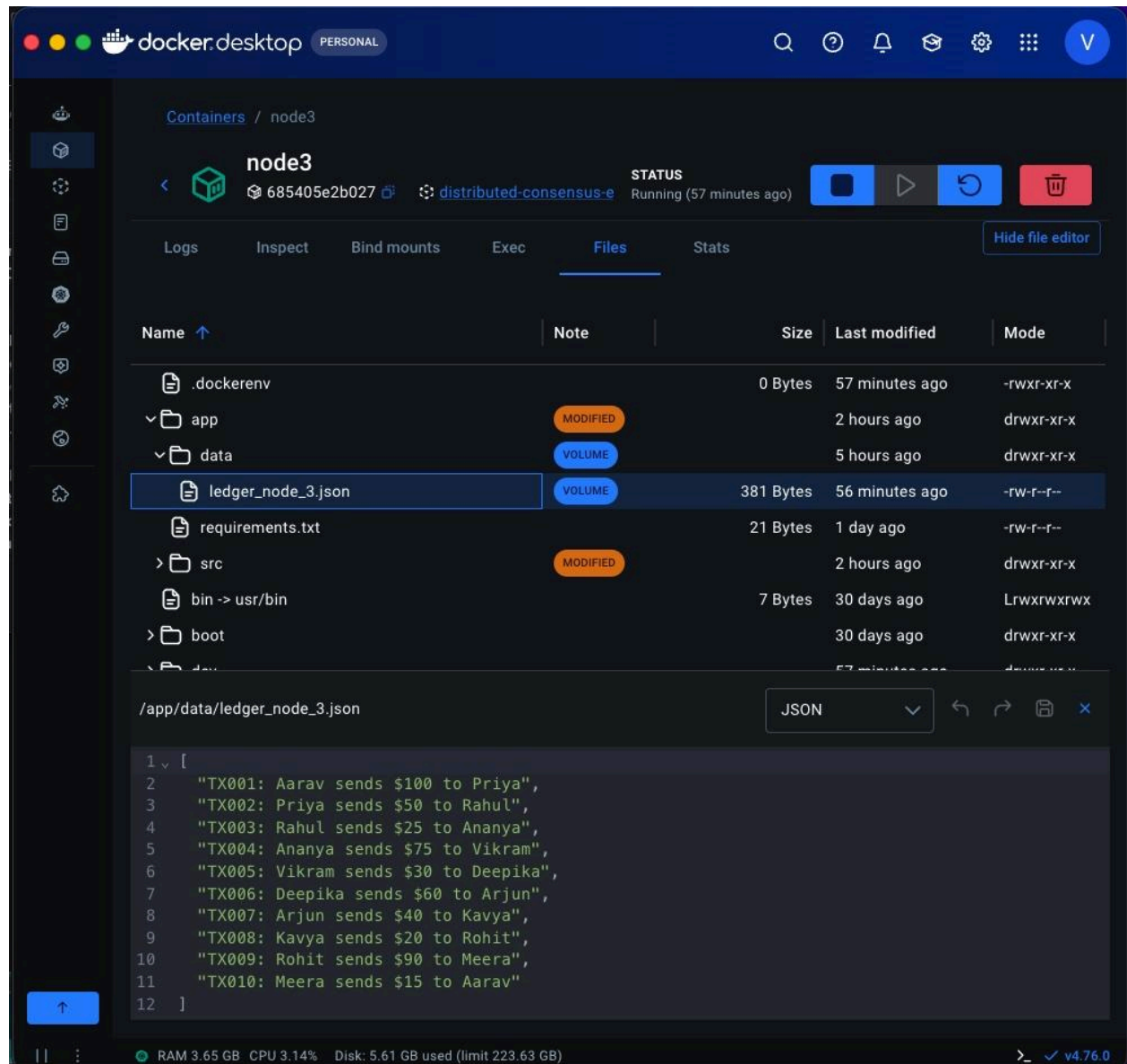
Name	Note	Size	Last modified	Mode
.dockerenv		0 Bytes	56 minutes ago	-rwxr-xr-x
✓ app	MODIFIED		2 hours ago	drwxr-xr-x
✓ data	VOLUME		4 hours ago	drwxr-xr-x
ledger_node_1.json	VOLUME	381 Bytes	55 minutes ago	-rw-r--r--
requirements.txt		21 Bytes	1 day ago	-rw-r--r--
> src	MODIFIED		2 hours ago	drwxr-xr-x
bin -> usr/bin		7 Bytes	30 days ago	Lrwxrwxrwx
> boot			30 days ago	drwxr-xr-x
> dev			56 minutes ago	drwxr-xr-x

/app/data/ledger_node_1.json JSON

```
1 [
2   "TX001: Aarav sends $100 to Priya",
3   "TX002: Priya sends $50 to Rahul",
4   "TX003: Rahul sends $25 to Ananya",
5   "TX004: Ananya sends $75 to Vikram",
6   "TX005: Vikram sends $30 to Deepika",
7   "TX006: Deepika sends $60 to Arjun",
8   "TX007: Arjun sends $40 to Kavya",
9   "TX008: Kavya sends $20 to Rohit",
10  "TX009: Rohit sends $90 to Meera",
11  "TX010: Meera sends $15 to Aarav"
12 ]
```

RAM 3.64 GB CPU 3.52% Disk: 5.61 GB used (limit 223.63 GB) v4.76.0

Ledger Node 3



Step 8: Leader Failure Simulation

Stop the leader node and observe automatic leader re-election.

```
docker-compose stop node5
```

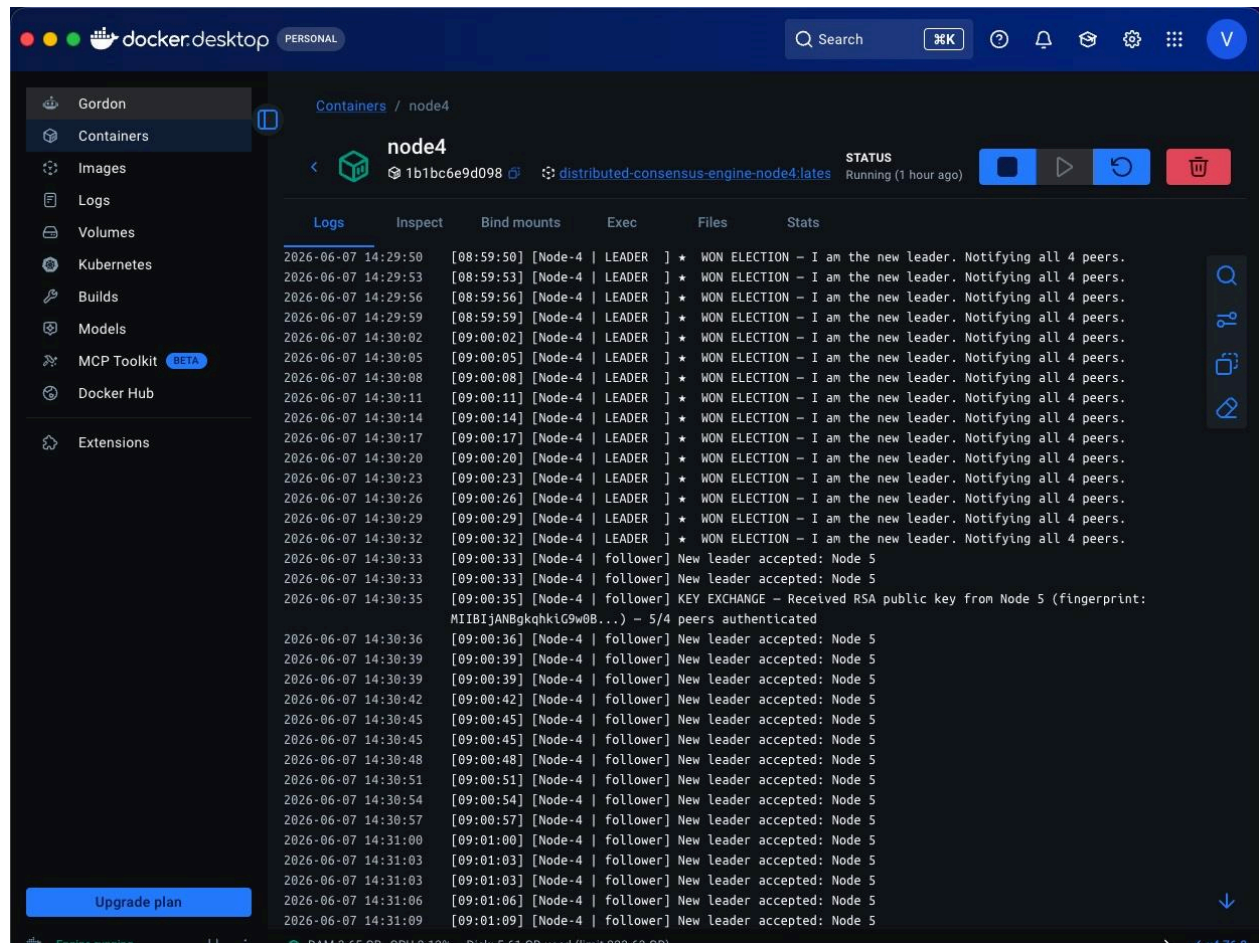
The screenshot shows the Docker Desktop interface with the 'Logs' tab selected. The logs are filtered by 'leader'. The log entries show a sequence of events where Node-4 becomes the new leader, followed by Node-1, Node-2, and Node-3 accepting it. Node-0 is shown as an attacker with no heartbeat, leading to a bully election. The logs are filtered by 'leader'.

Timestamp	Object	Message
2026-06-07 14:29:38	distributed-consensus-eng	[08:59:38] [Node-4 LEADER] * WON ELECTION - I am the new leader. Notifying all 4 peers.
2026-06-07 14:29:38	distributed-consensus-eng	[08:59:38] [Node-1 follower] New Leader accepted: Node 4
2026-06-07 14:29:38	distributed-consensus-eng	[08:59:38] [Node-2 follower] New Leader accepted: Node 4
2026-06-07 14:29:38	distributed-consensus-eng	[08:59:38] [Node-3 follower] New Leader accepted: Node 4
2026-06-07 14:29:39	distributed-consensus-eng	[08:59:39] [Node-0 ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...
2026-06-07 14:29:41	distributed-consensus-eng	[08:59:41] [Node-4 LEADER] * WON ELECTION - I am the new leader. Notifying all 4 peers.
2026-06-07 14:29:41	distributed-consensus-eng	[08:59:41] [Node-1 follower] New Leader accepted: Node 4
2026-06-07 14:29:41	distributed-consensus-eng	[08:59:41] [Node-2 follower] New Leader accepted: Node 4
2026-06-07 14:29:41	distributed-consensus-eng	[08:59:41] [Node-3 follower] New Leader accepted: Node 4
2026-06-07 14:29:42	distributed-consensus-eng	[08:59:42] [Node-0 ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...
2026-06-07 14:29:44	distributed-consensus-eng	[08:59:44] [Node-4 LEADER] * WON ELECTION - I am the new leader. Notifying all 4 peers.
2026-06-07 14:29:44	distributed-consensus-eng	[08:59:44] [Node-1 follower] New Leader accepted: Node 4
2026-06-07 14:29:44	distributed-consensus-eng	[08:59:44] [Node-2 follower] New Leader accepted: Node 4
2026-06-07 14:29:44	distributed-consensus-eng	[08:59:44] [Node-3 follower] New Leader accepted: Node 4

Step 9: Leader Recovery

Restart the failed leader and verify that it rejoins the cluster.

docker-compose start node5



Step 10: PBFT Mode Execution

Stop the cluster, change `MODE=paxos` to `MODE=pbft` in `docker-compose.yml`, and restart.

`docker-compose down`

`docker-compose up --build`

PERSONAL

Q

Search

31K

V

Logs

Give feedback

Q leader

Timestamp	Object	Message	
2026-06-07 15:33:57	distributed-consensus-eng	[10:03:57] [Node-2 follower] New leader accepted: Node 5	
2026-06-07 15:33:57	distributed-consensus-eng	[10:03:57] [Node-4 follower] New leader accepted: Node 5	
2026-06-07 15:33:57	distributed-consensus-eng	[10:03:57] [Node-3 follower] New leader accepted: Node 5	
2026-06-07 15:33:57	distributed-consensus-eng	[10:03:57] [Node-2 follower] New leader accepted: Node 5	
2026-06-07 15:33:57	distributed-consensus-eng	[10:03:57] [Node-4 follower] New leader accepted: Node 5	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [CLIENT] Leader discovered at node5:5000	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-5 LEADER] New transaction from client: 'TX001: Aarav sends \$100 to Priya' - starting PBFT consensus round	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-5 LEADER] PBFT Ph.1/3 - PRE-PREPARE: seq=1 assigned, SHA-256 digest computed, broadcasting to all.	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-5 LEADER] PBFT Ph.2/3 - Prepare quorum for seq=1 (3+ matching votes). Moving to Commit phase.	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-5 LEADER] Ledger updated - 1 total committed transactions on disk	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-5 LEADER] PBFT Ph.3 DONE - seq=1 COMMITTED (got 3+ commits). TX: 'TX001: Aarav sends \$100 to Priya'	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-0 ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-5 LEADER] * WON ELECTION - I am the new leader. Notifying all 4 peers.	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-2 follower] New leader accepted: Node 5	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-1 follower] New leader accepted: Node 5	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-4 follower] New leader accepted: Node 5	
2026-06-07 15:34:00	distributed-consensus-eng	[10:04:00] [Node-3 follower] New leader accepted: Node 5	
2026-06-07 15:34:03	distributed-consensus-eng	[10:04:03] [Node-0 ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...	
2026-06-07 15:34:03	distributed-consensus-eng	[10:04:03] [Node-5 LEADER] * WON ELECTION - I am the new leader. Notifying all 4 peers.	
2026-06-07 15:34:03	distributed-consensus-eng	[10:04:03] [Node-2 follower] New leader accepted: Node 5	
2026-06-07 15:34:03	distributed-consensus-eng	[10:04:03] [Node-1 follower] New leader accepted: Node 5	
2026-06-07 15:34:03	distributed-consensus-eng	[10:04:03] [Node-3 follower] New leader accepted: Node 5	
2026-06-07 15:34:03	distributed-consensus-eng	[10:04:03] [Node-4 follower] New leader accepted: Node 5	
2026-06-07 15:34:06	distributed-consensus-eng	[10:04:06] [Node-0 ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...	

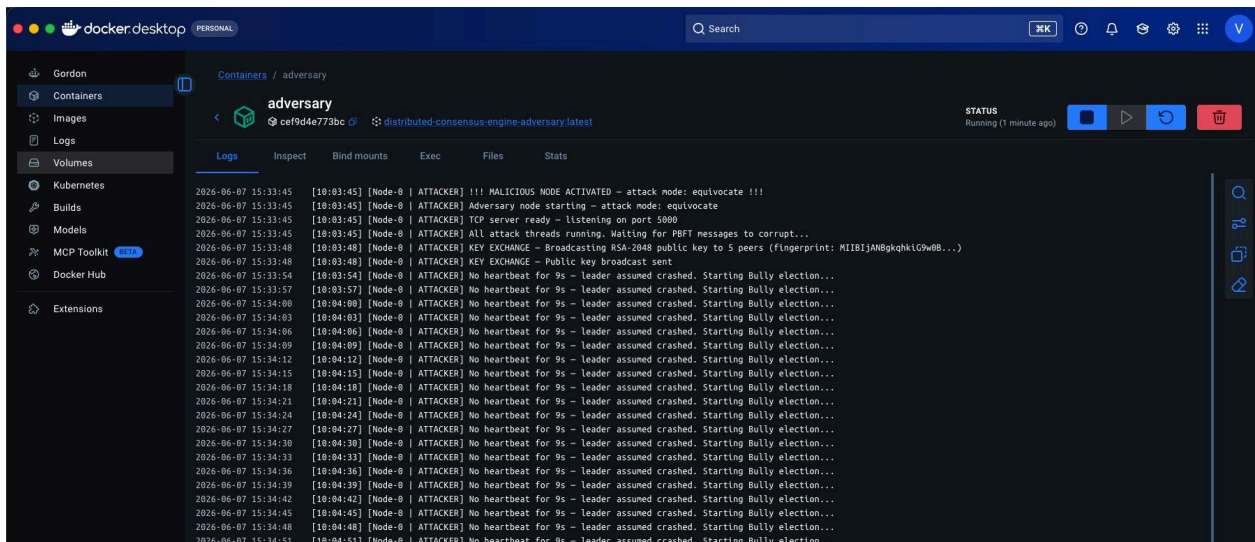
120 lines

Stick to bottom

Step 11: Byzantine Adversary Attack

Observe the malicious node sending conflicting messages.

docker logs adversary



Step 13: Chaos Testing

Execute the chaos testing script to simulate failures and network faults.

```
bash tests/chaos_test.sh
```



```
nodes checked with code 207
(base) vinaysingh@Vinays-Mac-mini distributed-consensus-engine % bash tests/chaos_test.sh
```

DISTRIBUTED CONSENSUS ENGINE - CHAOS TEST SUITE

--- TEST 0: Cluster Health Check ---

[INFO] Checking if all containers are running...

WARN[0000] /Users/vinaysingh/claude-projects/distributed-consensus-engine/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

[PASS] Cluster is running (7 services up)

[INFO] Waiting 10 seconds for cluster to stabilize...

--- TEST 1: Normal Operation (Baseline) ---

[INFO] Submitting transactions via client...

WARN[0000] /Users/vinaysingh/claude-projects/distributed-consensus-engine/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

[+] 5/5t 5/55

✓ Container node1	Running	0.0s
✓ Container node5	Running	0.0s
✓ Container node2	Running	0.0s
✓ Container node4	Running	0.0s
✓ Container node3	Running	0.0s

Container distributed-consensus-engine-client-run-6ddb30bc2f77 Creating

Container distributed-consensus-engine-client-run-6ddb30bc2f77 Created

TX 1: {'status': 'committed', 'transaction': 'BASELINE_TX_1'}

TX 2: {'status': 'committed', 'transaction': 'BASELINE_TX_2'}

TX 3: {'status': 'committed', 'transaction': 'BASELINE_TX_3'}

[INFO] Checking ledger consistency across nodes...

Node 1 ledger: [

"TX001: Aarav sends \$100 to Priya",
"TX002: Priya sends \$50 to Rahul",
"TX003: Rahul sends \$25 to Ananya",
"TX004: Ananya sends \$75 to Vikram",
"TX005: Vikram sends \$30 to Deepika",
"TX006: Deepika sends \$60 to Arjun",
"TX007: Arjun sends \$40 to Kavya",
"TX008: Kavya sends \$20 to Rohit",
"TX009: Rohit sends \$90 to Meera",
"TX010: Meera sends \$15 to Aarav",
"BASELINE_TX_1",
"BASELINE_TX_2",
"BASELINE_TX_3"

]

Node 2 ledger: [

"TX001: Aarav sends \$100 to Priya",
"TX002: Priya sends \$50 to Rahul",
"TX003: Rahul sends \$25 to Ananya",
"TX004: Ananya sends \$75 to Vikram",

> CHAT

```

--- TEST 4: Byzantine Fault Tolerance (PBFT) ---
[INFO] Adversary node is active with equivocation attack...
[INFO] Checking adversary logs for malicious activity...
adversary | [10:07:07] [Node-0 | ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...
adversary | [10:07:10] [Node-0 | ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...
adversary | [10:07:13] [Node-0 | ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...
adversary | [10:07:16] [Node-0 | ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...
adversary | [10:07:19] [Node-0 | ATTACKER] No heartbeat for 9s - leader assumed crashed. Starting Bully election...
[INFO] Submitting transaction - honest nodes should reach consensus despite adversary...
WARN[0000] /Users/vinaysingh/claude-projects/distributed-consensus-engine/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] 5/5t 5/55
✓ Container node4 Running 0.0s
✓ Container node1 Running 0.0s
✓ Container node3 Running 0.0s
✓ Container node2 Running 0.0s
✓ Container node5 Running 0.0s
Container distributed-consensus-engine-client-run-371c13f81740 Creating
Container distributed-consensus-engine-client-run-371c13f81740 Created
Result: {'status': 'failed', 'transaction': 'BYZANTINE_PROOF_TX'}
[INFO] Checking that adversary's fake messages were rejected...
time="2026-06-07T15:37:32+05:30" level=warning msg="/Users/vinaysingh/claude-projects/distributed-consensus-engine/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[PASS] PBFT consensus reached despite Byzantine adversary

--- TEST 5: High Latency Network ---
[INFO] Adding 500ms latency to node3 via Toxiproxy...
[INFO] Submitting transaction under high latency...
WARN[0000] /Users/vinaysingh/claude-projects/distributed-consensus-engine/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] 5/5t 5/55
✓ Container node5 Running 0.0s
✓ Container node3 Running 0.0s
✓ Container node4 Running 0.0s
✓ Container node2 Running 0.0s
✓ Container node1 Running 0.0s
Container distributed-consensus-engine-client-run-3b0c44bbabc0 Creating
Container distributed-consensus-engine-client-run-3b0c44bbabc0 Created
Result: {'status': 'failed', 'transaction': 'LATENCY_TEST_TX'} (took 5.02s)
[PASS] Consensus achieved despite network latency

=====
CHAOS TEST SUITE COMPLETE
=====

```

Step 14: Ledger Verification

Verify that all honest nodes contain identical committed transactions.

Node 4 Ledger

Containers / node4

node4

e82df6e29ee8 distributed-consensus-engine-node4:latest

STATUS
Running (7 minutes ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

Hide file editor

Name	Note	Size	Last modified	Mode
.dockerenv		0 Bytes	7 minutes ago	-rwxr-xr-x
app	MODIFIED		2 hours ago	drwxr-xr-x
data	VOLUME		5 hours ago	drwxr-xr-x
ledger_node_4.json	VOLUME	438 Bytes	4 minutes ago	-rw-r--
requirements.txt		21 Bytes	1 day ago	-rw-r--
src	MODIFIED		2 hours ago	drwxr-xr-x
bin -> usr/bin		7 Bytes	30 days ago	Lrwxrwxrwx
boot			30 days ago	drwxr-xr-x

/app/data/ledger_node_4.jsonJSON

```
1 [
2   "TX001: Aarav sends $100 to Priya",
3   "TX002: Priya sends $50 to Rahul",
4   "TX003: Rahul sends $25 to Ananya",
5   "TX004: Ananya sends $75 to Vikram",
6   "TX005: Vikram sends $30 to Deepika",
7   "TX006: Deepika sends $60 to Arjun",
8   "TX007: Arjun sends $40 to Kavya",
9   "TX008: Kavya sends $20 to Rohit",
10  "TX009: Rohit sends $90 to Meera",
11  "TX010: Meera sends $15 to Aarav",
12  "BASELINE_TX_1",
13  "BASELINE_TX_2",
```

Node 5 Ledger

Containers / node5

node5

939bf119df27 distributed-consensus-engine-node5:latest

STATUS
Running (4 minutes ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

Hide file editor

Name	Note	Size	Last modified	Mode
.dockerenv		0 Bytes	8 minutes ago	-rwxr-xr-x
app	MODIFIED		2 hours ago	drwxr-xr-x
data	VOLUME		5 hours ago	drwxr-xr-x
ledger_node_5.json	VOLUME	438 Bytes	5 minutes ago	-rw-r--
requirements.txt		21 Bytes	1 day ago	-rw-r--
src	MODIFIED		2 hours ago	drwxr-xr-x
bin -> usr/bin		7 Bytes	30 days ago	Lrwxrwxrwx
boot			30 days ago	drwxr-xr-x

/app/data/ledger_node_5.jsonJSON

```
1 [
2   "TX001: Aarav sends $100 to Priya",
3   "TX002: Priya sends $50 to Rahul",
4   "TX003: Rahul sends $25 to Ananya",
5   "TX004: Ananya sends $75 to Vikram",
6   "TX005: Vikram sends $30 to Deepika",
7   "TX006: Deepika sends $60 to Arjun",
8   "TX007: Arjun sends $40 to Kavya",
9   "TX008: Kavya sends $20 to Rohit",
10  "TX009: Rohit sends $90 to Meera",
11  "TX010: Meera sends $15 to Aarav",
12  "BASELINE_TX_1",
13  "BASELINE_TX_2",
```

Step 15: Shutdown the Cluster

Stop all running containers after testing is complete.

12. Conclusion

This project demonstrates the practical implementation of distributed consensus mechanisms using Python and Docker.

Leader Election ensures continuous coordination, Paxos provides crash fault tolerance, and PBFT enables Byzantine fault tolerance by detecting and rejecting malicious behavior.

The system maintains a consistent transaction ledger across distributed nodes even under failures and adverse network conditions, making it a strong example of reliable distributed system design.